
TÀI LIỆU ĐẶC TẢ KỸ THUẬT: HỆ THỐNG AUDIOBOOK KINH DỊ AI (GHOST STORY NARRATOR)

PHẦN 1: TỔNG QUAN KIẾN TRÚC & QUY TRÌNH DỮ LIỆU (DATA PIPELINE)

Hệ thống được thiết kế theo nguyên tắc: **Giữ nguyên kiến trúc âm học phức tạp của StyleTTS2 bản gốc (yl4579/StyleTTS2)** để khai thác sức mạnh của Diffusion, JAT (Joint Adversarial Training) và SLM Discriminator trên môi trường Windows 11 với 24GB VRAM. Đồng thời, đơn giản hóa tối đa quy trình gán nhãn biểu cảm và luồng Inference.

I. Cấu trúc Thư mục Dự án (Roadmap)

Dự án được tổ chức module hóa, cách ly giữa phần xử lý dữ liệu, huấn luyện và suy luận. Không sử dụng "monolithic pipeline", hãy chia ra các scripts để giúp resume hoặc debug từng bước rõ ràng.

TTS_StyleTTS2/

- | — fine-tune/
- | | — config/ ← 3 config stage + prepare_vivoice_config.yaml
- | | — data_pipeline/
- | | | — prepare_ngan/
- | | | — prepare_ood/
- | | | — prepare_vicoice/ ← 5 scripts + config.yaml
- | | — plbert/ (phiên bản cũ bị lỗi sẽ không sử dụng)
- | | — plbert_v2/ (Chứa mô hình plbert đã fine-tune cho tiếng Việt chuẩn cấu trúc)
- | | — train_wrapper.py
- | | — create_mean_style.py
- | | — nlp_generator.py
- | | — tts_generator.py
- | — StyleTTS2/ ← repo gốc

II. Quy trình Chuẩn bị Dữ liệu Huấn luyện (Data Pipeline)

Tất cả văn bản đi vào mô hình **bắt buộc phải ở dạng âm vị (phoneme)** thông qua thư viện `viphoneme`. Quá trình này được thực hiện qua các bước sau:

Bước 1: Chuẩn bị Dataset Nền (ViVoice) & Xây dựng Từ điển (prepare_vivoice)

- **Mục tiêu:** Xử lý bộ dữ liệu ViVoice từ Hugging Face thành format chuẩn âm vị, đồng thời xây dựng bộ từ điển ký tự (Vocabulary) để xác định chính xác tham số `n_token` cho mô hình.
- **Đầu vào:** Dataset [capleaf/viVoice](#) (chứa các file `.parquet`).

Định dạng gốc: Audio + text được **đóng gói trong file `.parquet`**

Theo dataset card:

Format: `parquet`

Features gồm: `audio`, `text`, `channel`

- **Các bước xử lý - pipeline cho viVoice:**

Bước 2.1 - Tải dữ liệu: Sử dụng thư viện `datasets` của Hugging Face để tải file dataset về ổ cứng local.

Bước 2.2 - Resampling & Normalization: Lặp qua từng file audio. Sử dụng `librosa` hoặc `torchaudio` để resample về đúng **24000Hz**, chuyển sang **Mono** (1 kênh), và format **16-bit PCM**. Lưu sang một thư mục mới (ví dụ: `vivoice_clean_wavs/`).

Bước 2.3 - Phonemization (Grapheme-to-Phoneme): Lặp qua các câu text trong data. Đưa chuỗi text qua thư viện `viphoneme`, sử dụng hàm `vi2IPA_split(text, " ")` để chuyển thành âm vị.

Ví dụ: Text thô: `xin chào` -> Viphoneme output: `k_ɿ̌_j_1 ɿ̌_i_2 n_wə_3`.

Bước 2.4 - Build Vocabulary: Quét qua toàn bộ chuỗi IPA vừa tạo, đếm tất cả các ký tự duy nhất (bao gồm cả khoảng trắng, dấu câu).

Bước 2.5 - Tạo Filelist: Tạo 2 file `train_list.txt` (95% data) và `val_list.txt` (5% data).

Cấu trúc 1 dòng chuẩn:

`Đường_dẫn_tuyệt_đối_đến_file.wav|chuỗi_âm_vị|speaker_id`
(`wav_path|phoneme|speaker_id`)

Ví dụ: `/data/vivoice_clean_wavs/0001.wav|s i n c a w f|0`

- **Đầu ra:**
 - Thư mục `vivoice_clean_wavs/`.

- File `vivoice_train_list.txt` và `vivoice_val_list.txt` theo định dạng:
Wav_path | phoneme | speaker_id
- File **phoneme_vocab.json** (Chứa mapping ID và tổng số n_token thực tế để cấu hình file YAML sau này).

Bước 2: Cập nhật Dataset Fine-tune (Giọng Bắc Ngạn) (`prepare_ngan_phoneme`)

- **Mục tiêu:** Đưa tập dữ liệu giọng bắc Ngạn (đã lọc sạch qua pipeline 7 bước với Demucs, DeepFilterNet, Pyannote, Whisper, DNSMOS) về chuẩn âm vị.
- **Đầu vào:** Thư mục `output_dataset/wavs/` (đã chuẩn 24kHz, -20 LUFS) và file `filelist_train.txt` (đang chứa text thô).
- **Xử lý:** Xóa các số, ký tự đặc biệt còn sót lại trong text, sau đó chạy qua `viphoneme.vi2IPA()`. Quét các ký tự IPA này để bổ sung/kiểm tra chéo với `phoneme_vocab.json` ở Bước 1.
- **Thêm speaker_id:** append thêm `|0` vào cuối mỗi dòng (Ví dụ:
`wavs/ngan_0001.wav|s_i_n_c_a_w_f|0`)
- **Đầu ra:** `ngan_train_phoneme.txt` và `ngan_val_phoneme.txt`.
- **Lưu ý:** Xóa các ký tự đặc biệt, số (nếu còn sót) trước khi đưa vào **viphoneme**.

Bước 3: Chuẩn bị Out-of-Domain Text (OOD Text) cho JAT (`prepare_ood_text`)

- **Mục tiêu:** Cung cấp văn bản không có audio đi kèm cho quá trình Joint Adversarial Training (JAT) ở Giai đoạn 2 và 3, giúp mô hình học ngữ điệu cho các từ vựng mới thuộc domain truyện ma.
- **Đầu vào:** File `raw_ood_texts.txt` chứa khoảng 50,000 câu truyện ma thô.
- **Các bước xử lý:**

Bước 3.1 - Clean Text: Viết script lọc bỏ các câu có chứa số (hoặc chuyển số thành chữ), bỏ các ngoặc kép, ngoặc đơn rườm rà. Chỉ giữ lại dấu `, . ? !`.

Bước 3.2 - Phonemization: Chạy qua **viphoneme** toàn bộ 50,000 câu này.

Bước 3.3 - Lưu file: Xuất ra file `OOD_texts_phoneme.txt`. Mỗi dòng là một chuỗi âm vị của 1 câu. (Lưu ý: File này không có đường dẫn audio `.wav`).

Bước 4: Huấn luyện Text Encoder (PL-BERT Tiếng Việt)

- **Mục tiêu:** Tạo mô hình ngôn ngữ hiểu ngữ nghĩa tiếng Việt ở cấp độ âm vị, thay thế cho PL-BERT tiếng Anh gốc.

- **Thực thi:**
 1. Gộp toàn bộ **text âm vị (chỉ gộp text)** từ ViVoice, tập Bác Ngạn và OOD Text thành file `all_corpus_phoneme.txt`.
 2. Sử dụng thư mục `Utils/PLBERT` trong repo gốc của StyleTTS2, cập nhật tham số vocabulary size bằng với con số `n_token` đã đếm được.
 3. Chạy script `train PL-BERT` từ đầu.
- **Đầu ra:** Các file checkpoint `stepxxx_vi.t7` để trở vào `config.yaml` của StyleTTS2.

Bước 5: Tải Mô hình Giám khảo (WavLM-Base-Plus)

- **Mục tiêu:** Cung cấp SLM Discriminator đánh giá mức độ "giống người" (human-like) của audio sinh ra.
- **Thực thi:** Tải trọng số pre-trained `pytorch_model.bin` của `microsoft/wavlm-base-plus` từ Hugging Face, đặt vào thư mục `Utils/` mà không cần fine-tune thêm.

PHẦN 2: MODULE NLP (QWEN) & LƯỒNG INFERENCE 2 PHASE

Luồng suy luận (Inference) thực tế sẽ KHÔNG chạy song song. Nó được tách làm hai tiến trình hoàn toàn độc lập, giao tiếp với nhau duy nhất qua một file `script.json`.

I. PHASE 1: Đạo diễn Kịch bản (Module NLP với Qwen3.5-27B-AWQ)

(<https://huggingface.co/QuantTrio/Qwen3.5-27B-AWQ>)

Mục tiêu: Sử dụng trí tuệ vượt trội của bản 27B để đọc hiểu văn bản, cắt câu hợp lý và chỉ định nhịp điệu kinh dị (khoảng lặng) mà không làm tốn VRAM của tiến trình TTS.

Sử dụng phương án thay thế nếu không thể infer trên GPU: Gọi API Gemini Flash free.

1. Khởi tạo Server API (Chạy thử công ở Terminal 1):

Do chúng ta dùng định dạng AWQ tối ưu cho GPU, bạn sẽ dùng `vLLM` để host mô hình thành một OpenAI-compatible API local. Lệnh này giới hạn Qwen chỉ được ăn tối đa khoảng 16-18GB VRAM:

Bash

```
python -m vllm.entrypoints.openai.api_server \
```

```
--model QuantTrio/Qwen3.5-27B-AWQ \
--quantization awq \
--max-model-len 4096 \
--gpu-memory-utilization 0.8 \
--port 8000
```

2. Script xử lý ngôn ngữ (nlp_generator.py):

- **Nhiệm vụ:** Đọc kịch bản truyện ma (.txt), đóng gói vào payload, gửi request (bằng thư viện requests hoặc openai) tới <http://localhost:8000/v1/chat/completions>.
- **Master Prompt cấu hình cho Qwen:**

Plaintext

Bạn là trợ lý AI chuyên phân tích kịch bản truyện ma tiếng Việt. Nhiệm vụ của bạn là xử lý văn bản thô và trích xuất thành MẢNG JSON.

Quy tắc:

1. Phân đoạn: Cắt đoạn dài thành các câu ngắn phù hợp (10-20 từ) để nhip thở tự nhiên.
2. Khoảng lặng (pause_after_ms), tuân theo các quy tắc để tạo không khí kinh dị:
 - Dấu phẩy (,): 200 - 300 ms.
 - Dấu (. ? !): 500 - 800 ms.
 - Chuyển cảnh, cao trào, rừng rợn: 1500 - 2500 ms.
3. Vai trò (role): "narrator" (lời dẫn) hoặc "male_character", "femal_character" (lời thoại nam hoặc nữ).

Output **BẮT BUỘC CHỈ LÀ MỘT MẢNG JSON**.

Định dạng object: {"id": int, "role": "string", "text": "string", "pause_after_ms": int}
KHÔNG in ra markdown, **KHÔNG** giải thích.

Mẫu Output JSON mong đợi (Đầu vào cho code Python):

JSON

```
[
{
```

```

    "id": 1,
    "role": "narrator",
    "text": "Đồng hồ điểm đúng mười hai giờ đêm.",
    "pause_after_ms": 1500
  },
  {
    "id": 2,
    "role": "narrator",
    "text": "Bỗng nhiên, từ ngoài hành lang vang lên tiếng bước chân.",
    "pause_after_ms": 2000
  },
  {
    "id": 3,
    "role": "male_character",
    "text": "Ai ở đó?",
    "pause_after_ms": 1000
  }
]

```

- **Đầu ra:** Code Python nhận JSON string, parse và lưu thành file script.json xuống ổ cứng.
- **GIẢI ĐOẠN DỌN DẸP (CRITICAL):** Sau khi có script.json, bạn **tắt hoàn toàn Terminal 1** (nhấn Ctrl+C tắt vLLM). Lúc này, GPU nhả lại nguyên vẹn 24GB VRAM cho Pha 2.

II. TRẠM TRUNG CHUYÊN: Chuẩn bị Phong cách (create_mean_style.py)

Đây là script chạy một lần duy nhất (offline) sau khi bạn đã train xong Giai đoạn 3 (Fine-tune Bác Ngạn) để chuẩn bị "nguyên liệu" cho Pha 2.

- **Mục tiêu:** Trích xuất **Mean Style Vector** (Vector phong cách trung bình) đại diện cho giọng đọc mặc mặc chuẩn mực nhất của nghệ sĩ Nguyễn Ngọc Ngạn.
- **Logic Code:**
 1. Load checkpoint StyleTTS2.
 2. Tạo một vòng lặp đọc filelist ngan_train_phoneme.txt, chọn ngẫu nhiên khoảng 100 audio dài và sạch nhất từ tập data bác Ngạn.
 3. Cho các audio này chạy qua bộ style_encoder của mô hình để thu về 100 tensor (thường có shape [1, 128]).

4. Dùng `torch.mean(tensors, dim=0)` để tính trung bình cộng.
5. Lưu kết quả ra file: `ngan_mean_style.pt`.

III. PHA 2: Thu âm & Hậu kỳ (Module TTS với StyleTTS2)

Mục tiêu: Đọc hiểu các nhãn nhịp điệu từ JSON, gọi StyleTTS2 sinh âm thanh thô, tự động tạo các mảng khoảng lặng xen kẽ để đẩy cảm xúc sợ hãi lên cao trào.

Script tổng hợp âm thanh (`tts_generator.py`):

Vì vLLM đã tắt, VRAM đang trống 24GB. StyleTTS2 (chỉ tốn khoảng 3-4GB lúc suy luận) sẽ chạy vô cùng mượt mà.

- **Bước 3.1 - Khởi tạo:** Load model StyleTTS2, load bộ từ điển `phoneme_vocab.json` (để đảm bảo `n_token` khớp 100%), load tensor `ngan_mean_style.pt`, và khởi tạo thư viện `viphoneme`.
- **Bước 3.2 - Đọc kịch bản:**

Đọc file `script.json`. Tạo một mảng lưu trữ: `audio_chunks = []`.

- **Bước 3.3 - Vòng lặp Sinh Âm thanh (Generation Loop):**

Lặp qua từng item trong JSON. Với mỗi item:

1. **Text 2 Phoneme:** `phonemes = viphoneme.vi2IPA(item["text"])`.
2. **Inference:** Gọi `model.inference(text=phonemes, ref_s=ngan_mean_style, alpha=0.3, beta=0.7)`. Kết quả thu được là một mảng numpy array 1 chiều chứa sóng âm thanh thô.
3. Lưu vào đệm: `audio_chunks.append(wav_array)`.

- **Bước 3.4 - Chèn Nhịp điệu (Silence Padding):**

Ngay sau bước sinh âm thanh của mỗi câu, tiến hành tạo khoảng lặng toán học:

1. Đọc giá trị khoảng lặng: `pause_ms = item["pause_after_ms"]`.
2. Tính số lượng sample: `num_samples = int((pause_ms / 1000.0) * 24000)`.
3. Tạo mảng im lặng tuyệt đối: `silence_array = np.zeros(num_samples, dtype=np.float32)`.
4. Lưu vào đệm: `audio_chunks.append(silence_array)`.

- **Bước 3.5 - Xuất file (Export):**

Sau khi vòng lặp kết thúc, mảng đệm sẽ có dạng: [Audio_Câu_1, Silence_1, Audio_Câu_2, Silence_2,...].

Dùng lệnh `final_wav = np.concatenate(audio_chunks)` để gộp tất cả lại thành một dải âm thanh duy nhất. Dùng thư viện `soundfile` lưu thành `output_ghost_story.wav`.

Tóm tắt Lợi ích của Kiến trúc này:

1. **An toàn bộ nhớ:** Loại bỏ 100% rủi ro đùng độ VRAM giữa hai "gã khổng lồ" Qwen 27B và StyleTTS2.
 2. **Khống chế cảm xúc tĩnh:** Không bắt mô hình TTS phải "cố gắng" tự rặn ra khoảng lặng (rất dễ bị nhiễu noise hoặc thờ ơ). Việc dùng mảng `np.zeros` để padding giúp khoảng lặng sạch sẽ tuyệt đối, mang lại cảm giác tĩnh mịch, ngọt ngào chuẩn xác của phim kinh dị.
-

PHẦN 3: CHIẾN LƯỢC HUẤN LUYỆN 3 GIAI ĐOẠN & WRAPPER QUẢN LÝ

Chiến thuật huấn luyện ở đây là "Đứng trên vai người khổng lồ": Dạy hệ thống phát âm tiếng Việt chuẩn xác trước, sau đó dạy biểu cảm (JAT/SLM), và cuối cùng mới thêm "tâm hồn" kịch bản truyện ma của nghệ sĩ Ngọc Ngạn vào.

I. Kiến trúc Quản lý Huấn luyện (`train_wrapper.py`)

Do thuật toán MAS (Monotonic Alignment Search) và JAT của StyleTTS2 rất phức tạp, việc viết lại một vòng lặp huấn luyện từ đầu (Custom Training Loop) mang rủi ro lỗi toán học cực cao.

Thay vào đó, hệ thống sử dụng một script **Wrapper/Manager** (`train_wrapper.py`):

- **Nhiệm vụ:** Đóng vai trò như một "nhạc trưởng" điều phối. Nó đọc tham số dòng lệnh (vd: `--stage 1`), sau đó tự động dùng thư viện `subprocess` để gọi ngầm các file script gốc của tác giả (`train_first.py`, `train_second.py`, `train_finetune.py`).
- **Đồng bộ Vocabulary:** Wrapper sẽ đọc file `phoneme_vocab.json` (tạo ra từ Bước 1 của Data Pipeline), lấy giá trị `n_token` thực tế và tự động tiêm (inject) vào cả 3 file config trước khi chạy. Điều này chống lỗi `IndexError` 100%.
- **Auto-Chaining (Bàn giao Checkpoint):** Wrapper tự động quét thư mục log của Giai đoạn trước, tìm file checkpoint có loss thấp nhất (hoặc mới nhất), và tự động điền đường dẫn đó vào tham số `pretrained_model` của Giai đoạn sau.

II. Giai đoạn 1: Base Model - Học Phát âm Tiếng Việt (Acoustic & Alignment)

- **Mục tiêu:** Mô hình chỉ tập trung học cách alignment giữa chuỗi âm vị (phoneme) và biểu diễn âm thanh (mel-spectrogram) để phát âm tròn vành rõ chữ.
- **Script thực thi:** Gọi ngàm train_first.py.
- **Cấu hình (config_stage1.yaml - Dựa trên schema config.yml gốc):**
 - **Dữ liệu:** Trỏ train_data và val_data tới filelist của tập ViVoice (đã chuyển âm vị).
 - **JAT & SLM (Tắt hoàn toàn):** Đặt tham số joint_epoch: 9999 (một epoch không bao giờ đạt tới) để mô hình không kích hoạt module biểu cảm, tập trung 100% vào phát âm cơ bản.
 - **Batch Size:** Vì tắt JAT, VRAM 24GB cực kỳ dư dả. Đẩy batch_size lên **16 hoặc 32** để mô hình hội tụ nhanh chóng.
 - **Learning Rate:** Giữ mức cơ bản của pre-train (thường là 1e-4). Có thể dùng chiến thuật warm-up (lr_scheduler_size, warmup_steps, warmup_ratio)
- **Tiêu chí dừng:** Dừng khi mel_loss và mono_align_loss giảm đều và chững lại. Có thể cân nhắc dùng chiến thuật early stop.

III. Giai đoạn 2: Expressive Training - Đánh thức Cảm xúc

- **Mục tiêu:** Bật các module nặng nhất của StyleTTS2. Sử dụng OOD Text để dạy Prosodic Encoder cách dự đoán nhịp điệu cho các từ vựng mới, và dùng WavLM để ép âm thanh sinh ra phải "giống người" (human-like).
- **Script thực thi:** Gọi ngàm train_second.py.
- **Cấu hình (config_stage2.yaml - Dựa trên schema config.yml gốc):**
 - **Dữ liệu:** Vẫn là tập ViVoice.
 - **OOD Text:** Trỏ đường dẫn ood_data tới file OOD_texts_phoneme.txt.
 - **JAT & SLM (Bật ngay lập tức):** Chỉnh joint_epoch: 0 và diff_epoch: 0. Đảm bảo đường dẫn tới trọng số WavLM-Base-Plus đã được trỏ đúng.
 - **Batch Size (CỰC KỲ QUAN TRỌNG):** Giảm batch_size xuống mức **4 hoặc 6 (có thể cân nhắc sử dụng gradient_accumulation_steps)**. Việc chạy đồng thời Text-to-Wave, OOD Inference, và bộ giám khảo WavLM sẽ ngốn VRAM khủng khiếp.

- **Pre-trained:** Tự động load checkpoint tốt nhất từ Giai đoạn 1.

IV. Giai đoạn 3: Fine-tune Giọng Bác Ngạn (The Ghost Story Soul)

- **Mục tiêu:** Chuyển đổi âm sắc (timbre) từ giọng ViVoice đa dạng sang duy nhất một chất giọng trầm ấm, rùng rợn của nghệ sĩ Nguyễn Ngọc Ngạn.
- **Script thực thi:** Gọi ngầm `train_finetune.py`.
- **Cấu hình (`config_stage3.yaml` - Dựa trên schema `config_ft.yml` gốc):**

 - **Dữ liệu:** Đổi `train_data` và `val_data` sang filelist của bác Ngạn (`ngan_train_phoneme.txt`).
 - **OOD Text & JAT/SLM:** Vẫn giữ BẬT để duy trì độ biểu cảm.
 - **Tham số tải trọng số:** Đặt `load_only_params: true` để chỉ lấy trọng số cốt lõi từ Giai đoạn 2, làm mới lại bộ Optimizer.
 - **Learning Rate (Bảo vệ trí nhớ):** Giảm LR xuống cực kỳ thấp, `ft_lr: 0.00001` ($1e-5$). Điều này giúp mô hình không bị "Catastrophic Forgetting" (quên cách phát âm tiếng Việt chuẩn nếu data bác Ngạn có nhiều hoặc ngắt nghỉ lạ).

- **Tiêu chí dừng:** Fine-tune không cần chạy quá nhiều epoch. Cứ mỗi 5-10 epoch, trích xuất checkpoint, chạy script `tts_generator.py` (Pha 2) với một câu thoại mẫu. Khi nào nghe đủ độ trầm và rùng rợn thì dừng quá trình huấn luyện.

V. Đánh giá & Kiểm thử (Evaluation)

Sau khi kết thúc Giai đoạn 3 và chốt được mô hình:

1. **MOS (Mean Opinion Score):** Dùng Qwen sinh một đoạn kịch bản 5 câu có nhịp điệu. Render âm thanh và mời người nghe đánh giá trên thang 1-5 về "Độ rùng rợn" và "Tính tự nhiên".
 2. **Kiểm tra phần cứng:** Ghi nhận log về mức VRAM cao nhất (Peak VRAM) trong Pha 1 (vLLM) và Pha 2 (StyleTTS2) để chứng minh tính hiệu quả của chiến lược cách ly tiến trình.
-